MHP
A PORSCHE COMPANY

GIT: CONCEPTE FUNDAMENTALE ȘI UTILIZARE PRACTICĂ

ANDREA TECAR
DARIUS FLOREA
DENISA BIGIU
SERGIU LIMBOI

AGENDA

01 Evoluția sistemelor de versionare

02 Ce este Git?

03 Cum colaborăm prin Git?

04 Best practices & mai mult Git

05 Demo

© MHP Management- und IT-Beratung GmbH

Înainte să începem...

Scanați codul QR
sau
intrați pe **menti.com**
și
introduceți codul **8349 1809**

Mentimeter

Formular înrolare voluntariat Git



Evoluția sistemelor de versionare

Cum gestionăm mai multe versiuni ale aceluiași proiect?



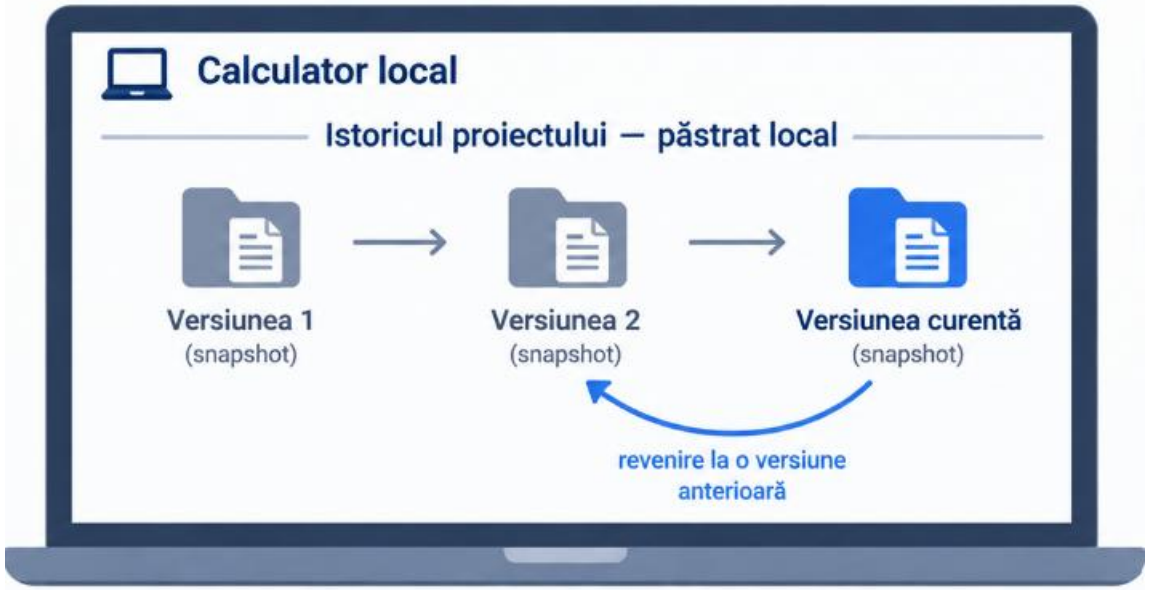
- Nu știm care este versiunea corectă
- Nu vedem exact ce s-a modificat
- Putem pierde sau suprascrie schimbări
- Colaborarea devine dificilă
- Revenirea la o versiune anterioară este complicată

Avem nevoie de un sistem care să păstreze istoricul modificărilor!

© MHP Management- und IT-Beratung GmbH

Versionare locală

- Versiunile proiectului sunt păstrate pe calculatorul local
- Fiecare versiune reprezintă un anumit moment din evoluția proiectului
- Putem reveni la o versiune anterioară
- Este suficient când lucrăm singuri, dar nu și când lucrăm în echipă

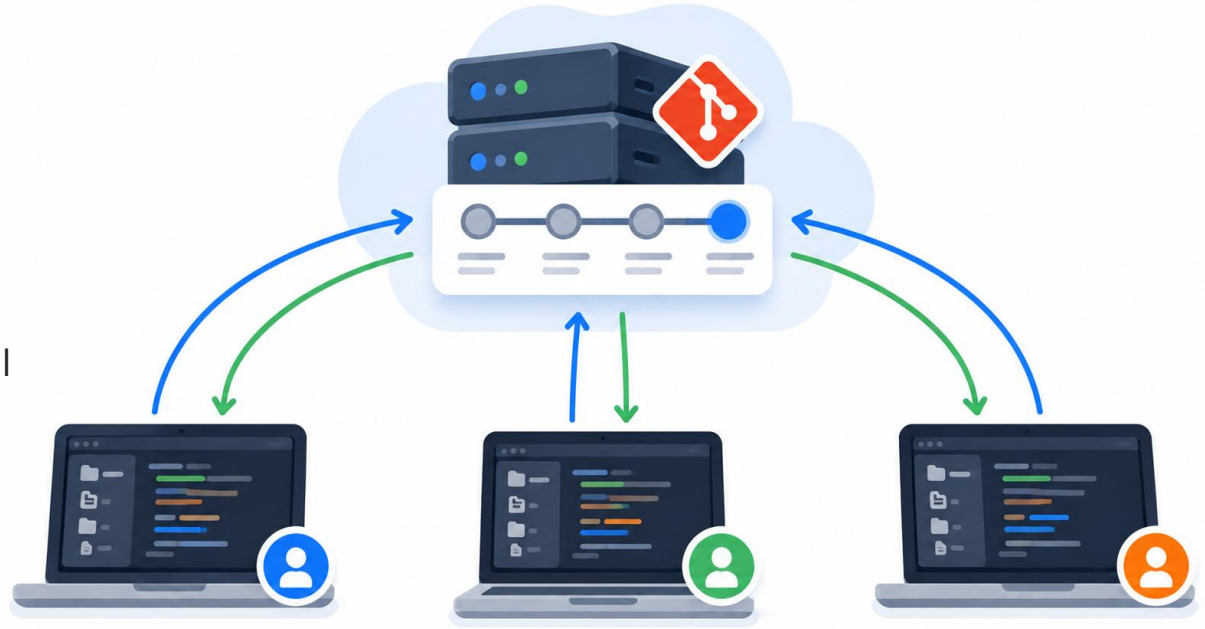


„Ce se întâmplă dacă se strică laptopul?"

© MHP Management- und IT-Beratung GmbH

Versionare centralizată

- Istoricul proiectului este păstrat pe un server central
- Membrii echipei lucrează asupra aceleiași versiuni
- Modificările sunt gestionate într-un singur loc
- Colaborarea devine mai simplă, dar echipa depinde de serverul central

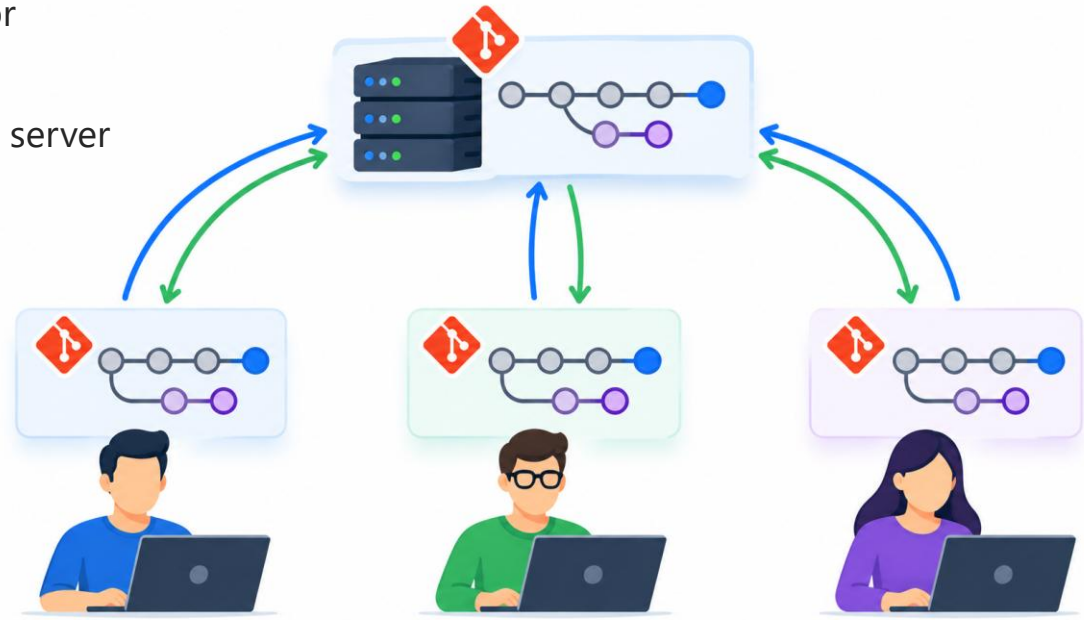


„Putem continua să lucrăm dacă nu avem acces la server?”

© MHP Management- und IT-Beratung GmbH

Versionare distribuită

- Fiecare dezvoltator are o copie completă a proiectului și a istoricului versiunilor
- Modificările pot fi realizate și înregistrate local, fără a fi necesară conectarea la server
- Copiile dezvoltatorilor pot fi sincronizate ulterior printr-un repository remote
- Fiecare dezvoltator deține propriul repository, cu propriul istoric complet
- Putem dezvolta și testa modificări independent



„Dacă serverul remote devine indisponibil, mai putem lucra la proiect?”

De la versionare locală la versionare distribuită

Model	Unde este istoricul?	Colaborare	Exemplu
Local	Pe calculatorul utilizatorului	Dificilă	Copii locale
Centralizat	Pe un server central	Posibilă prin server	SVN
Distribuit	Local și remote	Flexibilă	Git

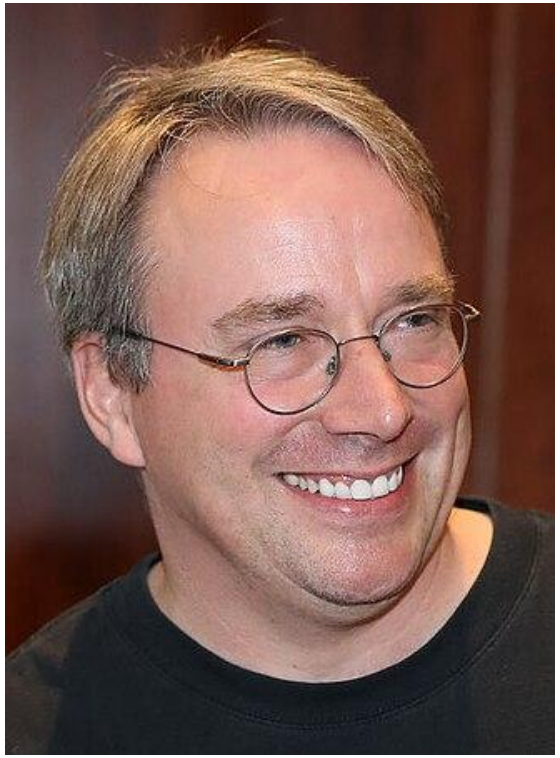
Git este un sistem distribuit de versionare.

© MHP Management- und IT-Beratung GmbH

Ce este Git?

Ce este Git?

- Git a fost creat de **Linus Torvalds** (care a creat și nucleul Linux) în 2005
- Linus Torvalds a ales numele „git” pentru prima versiune a instrumentului. În glumă, l-a descris drept *„the stupid content tracker”*
- Potrivit explicației sale, numele poate fi interpretat în mai multe moduri:
  - O combinație aleatorie de trei litere, ușor de pronunțat și nefolosită de o comandă UNIX obișnuită
  - Un cuvânt care, în engleza colocvială, poate însemna o persoană nesuferită sau proastă
  - O posibilă pronunție greșită a cuvântului „get”



Ce este Git?

- **Git este un sistem distribuit de control al versiunilor** care ne permite să urmărim, să înregistrăm și să gestionăm modificările unui proiect în timp.

- **Ce ne permite să facem?**
  - **SCM (Source Code Management)** – gestionarea codului sursă și a modificărilor acestuia
  - **Versionare** – păstrarea istoricului modificărilor și posibilitatea de a reveni la versiuni anterioare
  - **Dezvoltare în paralel** – putem lucra la funcționalități diferite fără a modifica direct aceeași versiune de lucru
  - **Colaborare** – putem integra modificările realizate de mai mulți dezvoltatori



- **Caracteristici esențiale**
  - Este **distribuit**: fiecare repository local conține istoricul complet al proiectului
  - Este **rapid și eficient** în gestionarea modificărilor
  - Funcționează local, fără a necesita permanent un server

© MHP Management- und IT-Beratung GmbH

Modelul mental Git

- Git gestionează modificările proiectului prin trei zone locale distincte:
  - **Working Directory** – fișierele proiectului la care lucrăm și pe care le modificăm
  - **Staging Area (Index)** – zona în care selectăm modificările care vor fi incluse în următoarea versiune
  - **Local Repository** – zona în care sunt înregistrate versiunile proiectului și istoricul acestora
- **Repository-ul remote** permite partajarea și sincronizarea istoricului cu alți dezvoltatori



© MHP Management- und IT-Beratung GmbH

Cum evoluează un proiect?

- Pe parcursul dezvoltării, fișierele unui proiect se modifică în mod repetat
  - **Modificăm fișierele** – adăugăm funcționalități, corectăm erori sau ștergem cod
  - **Selectăm modificările** – alegem ce schimbări dorim să includem în următoarea versiune
  - **Înregistrăm o versiune** – păstrăm un nou punct de referință în istoricul proiectului



- Git ne permite să controlăm aceste etape și să păstrăm istoricul evoluției proiectului

Dacă modificăm cinci fișiere, suntem obligați să includem toate modificările în următoarea versiune?

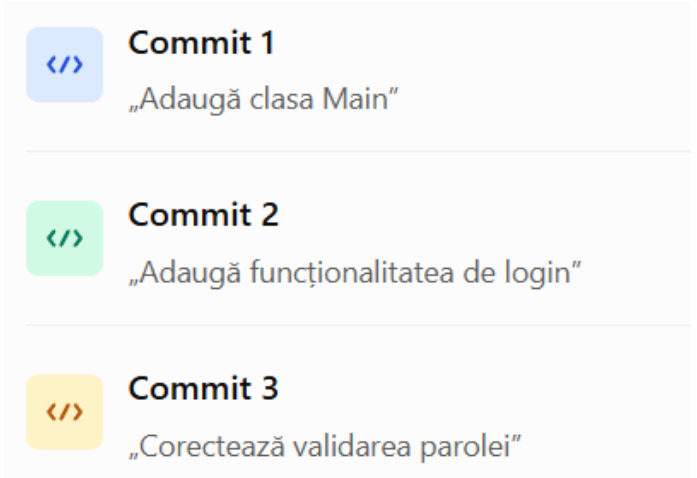
Ce este un commit?

- Un **commit** reprezintă o înregistrare a unui set de modificări în istoricul unui repository Git

- Fiecare commit marchează un punct în evoluția proiectului și ne permite să identificăm ce modificări au fost înregistrate

- **Ce conține un commit?**
  - Modificările înregistrate – schimbările selectate pentru acea versiune
  - Autorul – persoana care a realizat commit-ul
  - Data și ora – momentul înregistrării
  - Mesajul commit-ului – o descriere scurtă a modificărilor

- **De ce folosim commit-uri?**
  - Păstrăm istoricul evoluției proiectului
  - Putem identifica modificările introduse în timp
  - Putem reveni la versiuni anterioare ale proiectului



© MHP Management- und IT-Beratung GmbH

Cum creăm un commit?

- Pentru a înregistra modificările în istoricul Git, parcurgem trei pași:
  - Modificăm fișierele → Lucrăm în *Working Directory*
  - Selectăm modificările → Folosim ***git add*** pentru a selecta modificările care vor fi incluse în următorul commit și pentru a le adăuga în *Staging Area*
  - Înregistrăm modificările → Folosim ***git commit*** pentru a înregistra modificările selectate în istoricul *Local Repository*
  - Commit-ul este local; nu este trimis automat la server



Dacă modificăm **Main.java**, executăm **git add Main.java**, apoi modificăm din nou fișierul, ce modificare va ajunge în commit dacă executăm imediat **git commit**?

© MHP Management- und IT-Beratung GmbH

Unde găzduim repository-urile Git?

- **GitHub**
  - https://github.com/
  - Una dintre cele mai populare platforme pentru găzduirea proiectelor Git, colaborare și dezvoltare open-source



- **GitLab**
  - https://gitlab.com/
  - Integrează gestionarea codului sursă, colaborarea, securitatea și automatizarea proceselor de dezvoltare software



- **BitBucket**
  - https://bitbucket.org/product/
  - Platformă de găzduire a repository-urilor Git, integrată cu instrumentele Atlassian pentru gestionarea proiectelor și colaborare



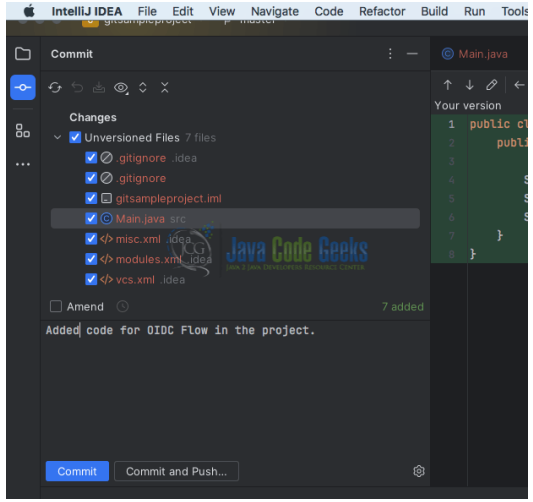
© MHP Management- und IT-Beratung GmbH

Tool-uri pentru lucrul cu Git

- **Terminal (CMD / GitBash)**
  - Permite executarea directă a comenzilor Git
  - Oferă acces la toate operațiile Git
  - Recomandat pentru înțelegerea comenzilor și a modului de funcționare Git



- **IntelliJ IDEA – Git Integration**
  - Integrează operațiile Git direct în mediul de dezvoltare
  - Permite gestionarea branch-urilor, commit-urilor, merge-urilor și conflictelor
  - Oferă acces la istoricul modificărilor și compararea versiunilor



- **Sourcetree**
  - Interfață grafică pentru gestionarea repository-urilor Git
  - Permite vizualizarea istoricului, branch-urilor și relațiilor dintre commit-uri
  - Facilitează operațiile Git fără introducerea manuală a comenzilor

Sourcetree

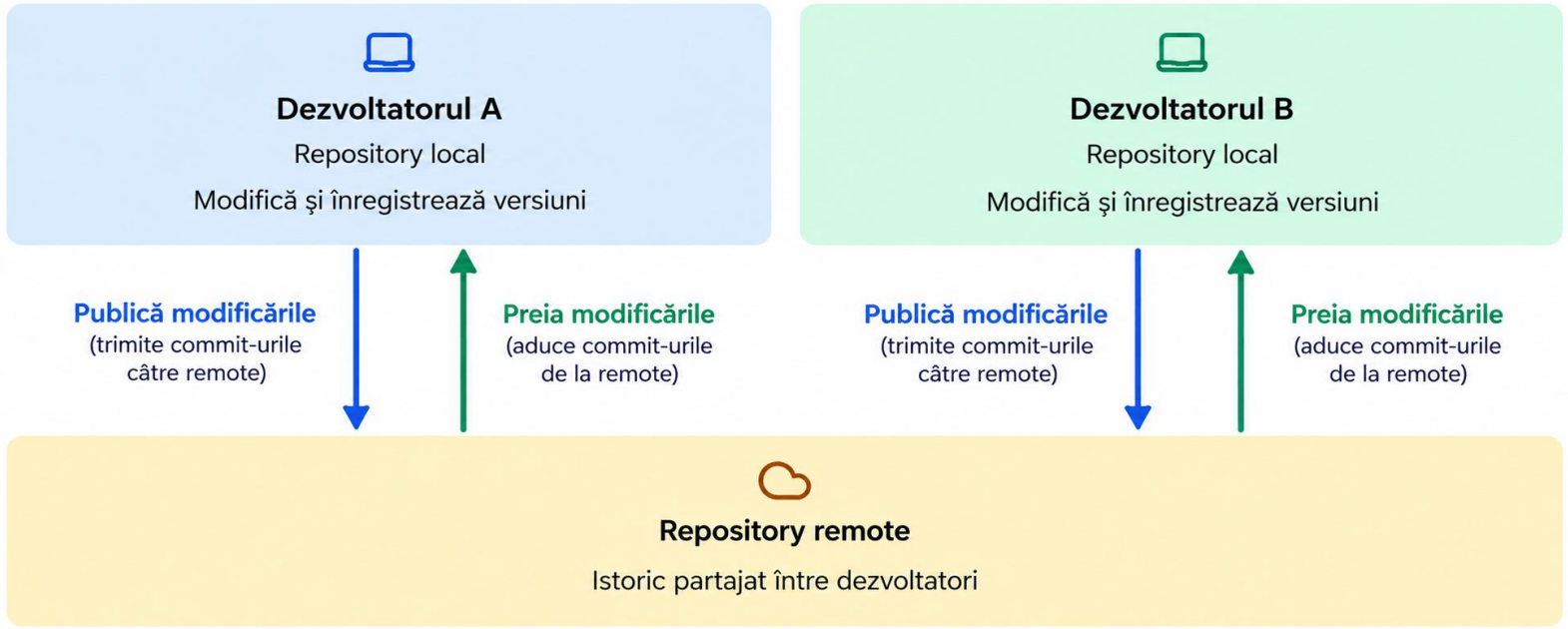
Cum colaborăm prin Git?

Cum colaborăm prin Git?

- Doi sau mai mulți dezvoltatori pot lucra la același proiect, chiar dacă folosesc calculatoare diferite.

- **Cum funcționează?**
  - Obținem o copie locală a proiectului – fiecare dezvoltator are propriul repository local
  - Lucrăm independent – modificăm fișierele și înregistrăm versiunile local
  - Partajăm modificările – trimitem commit-urile către repository-ul remote
  - Preluăm modificările colegilor – aducem în repository-ul local schimbările publicate de ceilalți

- **De ce este util?**
  - Dezvoltatorii pot lucra în paralel
  - Fiecare are propriul istoric local
  - Modificările pot fi partajate și integrate
  - Conflictele pot fi identificate și rezolvate



© MHP Management- und IT-Beratung GmbH

Cum obținem o copie locală a proiectului?

- Comanda **git clone** creează o copie locală a unui repository existent.

```
git clone https://github.com/user/proiect.git
```

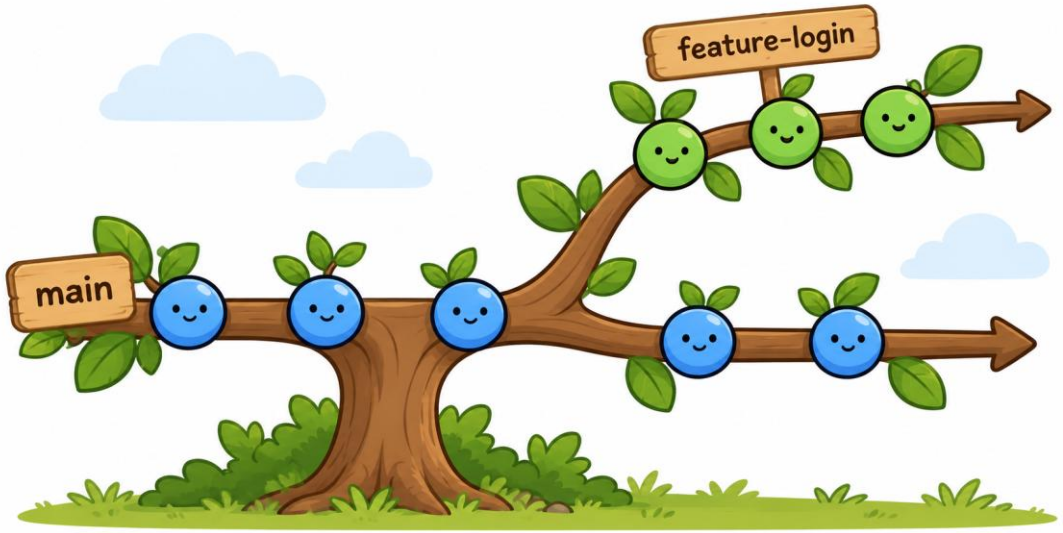
- **Ce se întâmplă?**
  - Se copiază fișierele proiectului pe calculatorul nostru
  - Se obține istoricul existent al proiectului
  - Se configurează conexiunea către repository-ul remote, pentru sincronizări ulterioare
  - Comanda creează un director local pentru proiect și descarcă repository-ul în acesta
  - **git clone** ne permite să începem lucrul la un proiect existent, păstrând istoricul său Git

„Dacă am clonat un repository, mai avem nevoie de conexiune la internet pentru a modifica fișierele și a crea commit-uri locale?”

© MHP Management- und IT-Beratung GmbH

Ce este un branch?

- Un **branch (ramură)** reprezintă o linie independentă de dezvoltare în cadrul unui repository Git
- Permite dezvoltarea unei funcționalități fără a modifica direct branch-ul principal
- Mai mulți dezvoltatori pot lucra în paralel la funcționalități diferite
- Modificările pot fi integrate ulterior în branch-ul principal

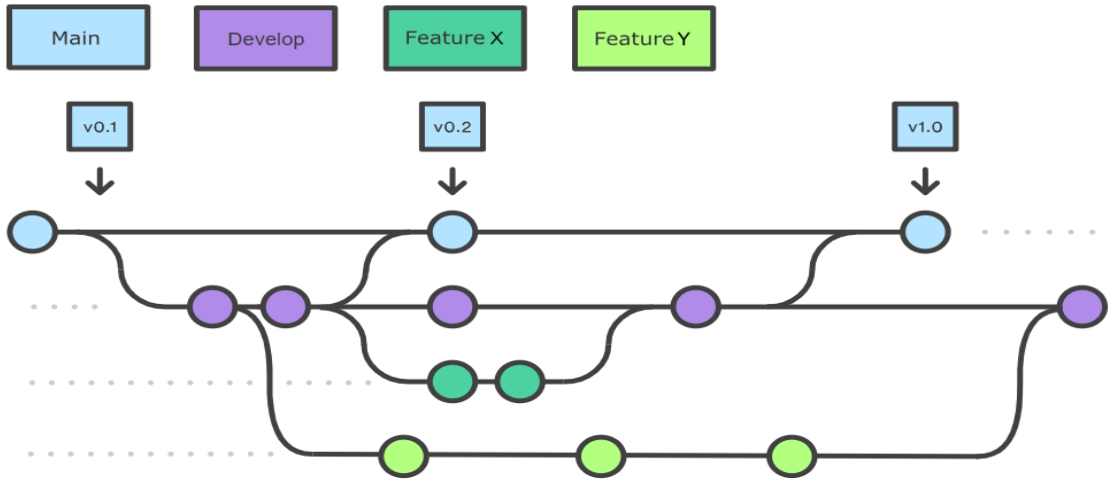


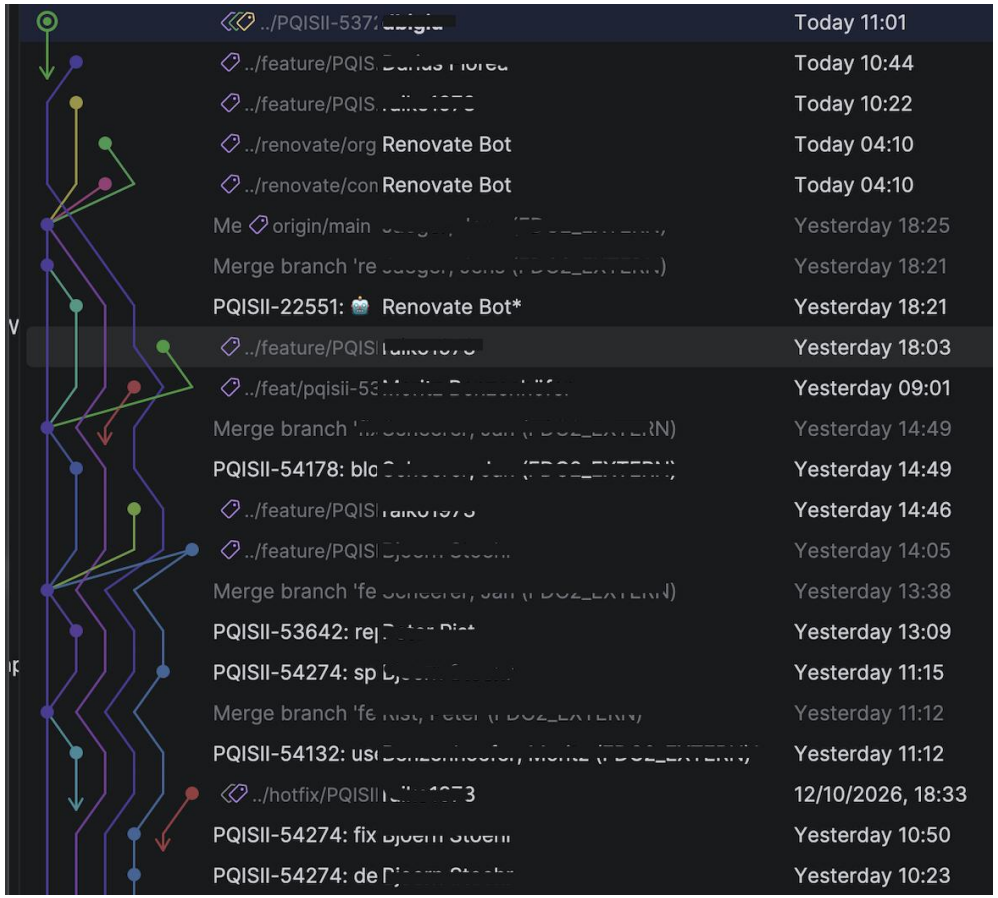
- **Exemplu**
  - main – branch-ul principal al proiectului
  - Feature-login – branch pentru implementarea autentificării
  - bugfix-validation – branch pentru corectarea unei erori

- **Cum creăm un branch?**
  - **git branch feature-login** (Creează un branch numit feature-login, fără să ne mute automat pe el)
  - **git checkout -b feature-login** (Creează branch-ul și trece pe el)
  - **git switch -c feature-login** (Creează branch-ul și trece pe el; aceasta este varianta modernă recomandată)

© MHP Management- und IT-Beratung GmbH

Workflow-ul obișnuit de software development în Git





© MHP Management- und IT-Beratung GmbH

Cum publicăm modificările?

- Comanda **git push** trimite commit-urile locale către un repository remote

- **Cum funcționează?**
  - Lucrăm la proiect în repository-ul local
  - Înregistrăm modificările prin unul sau mai multe commit-uri
  - Folosim **git push** pentru a publica aceste commit-uri pe repository-ul remote
  - Colegii pot prelua ulterior modificările publicate

- **git push -u origin feature-login**
  - origin – numele uzual al repository-ului remote
  - feature-login – branch-ul pe care îl publicăm
  - -u – configurează asocierea cu ramura remote corespunzătoare, pentru a simplifica comenzile ulterioare

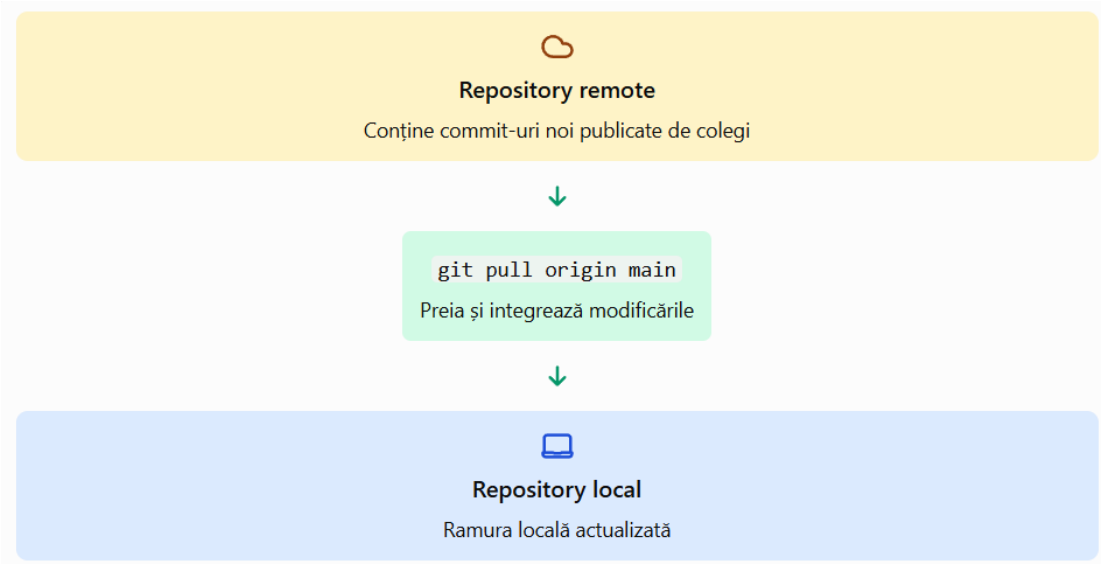


© MHP Management- und IT-Beratung GmbH

Cum preluăm modificările?

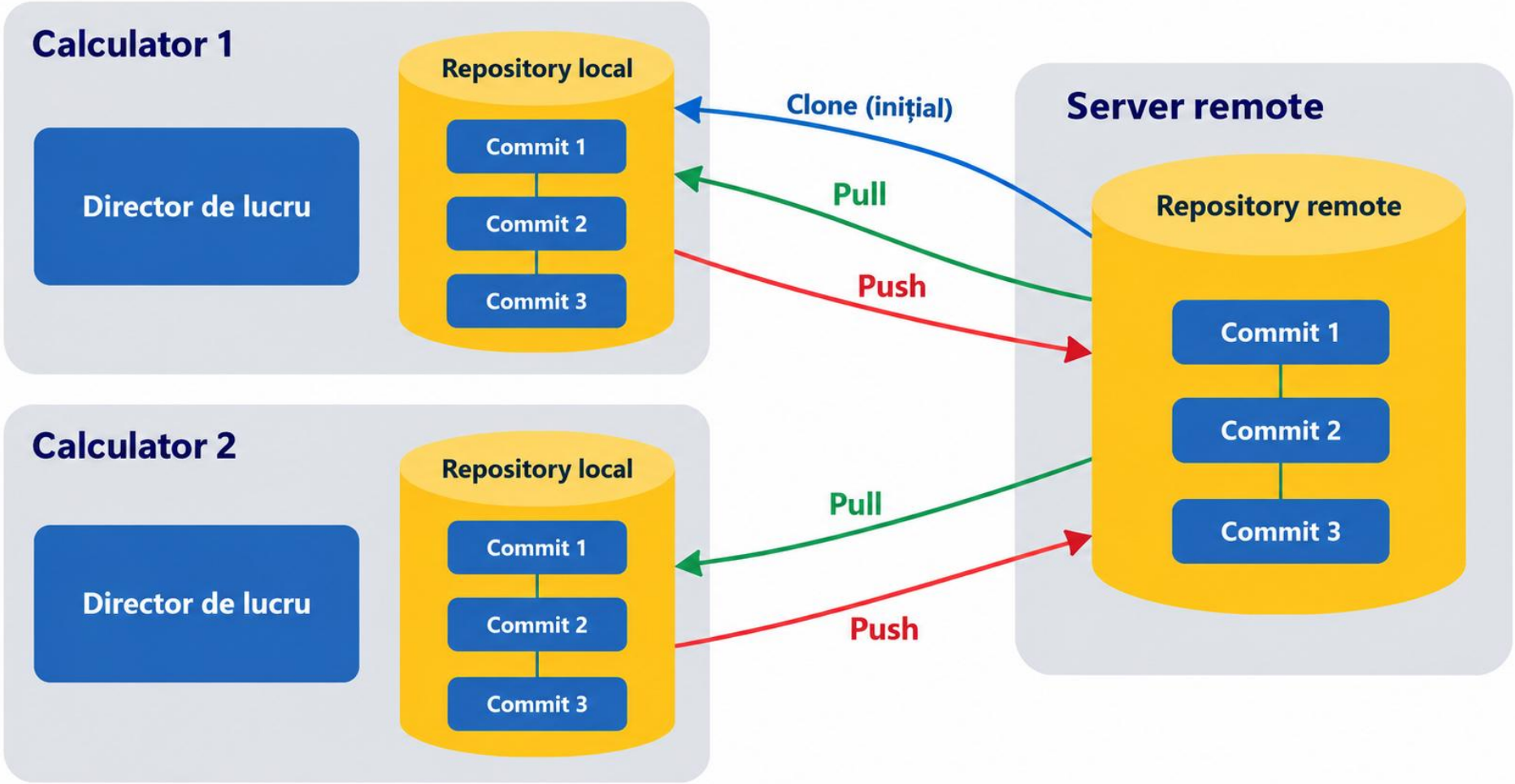
- Comanda **git pull** preia modificările dintr-un repository remote și încearcă să le integreze în ramura locală curentă

- **Cum funcționează?**
  - Un coleg publică modificări pe repository-ul remote
  - Noi executăm git pull pentru a prelua modificările
  - Git descarcă modificările noi și încearcă să le integreze în ramura locală
  - Dacă modificările sunt compatibile, putem continua lucrul cu versiunea actualizată



- **git pull origin main**
  - origin – numele repository-ului remote
  - main – branch-ul remote ale cărui modificări dorim să le integrăm

Cum se sincronizează repository-urile Git?



Best practices & mai mult Git

© MHP Management- und IT-Beratung GmbH

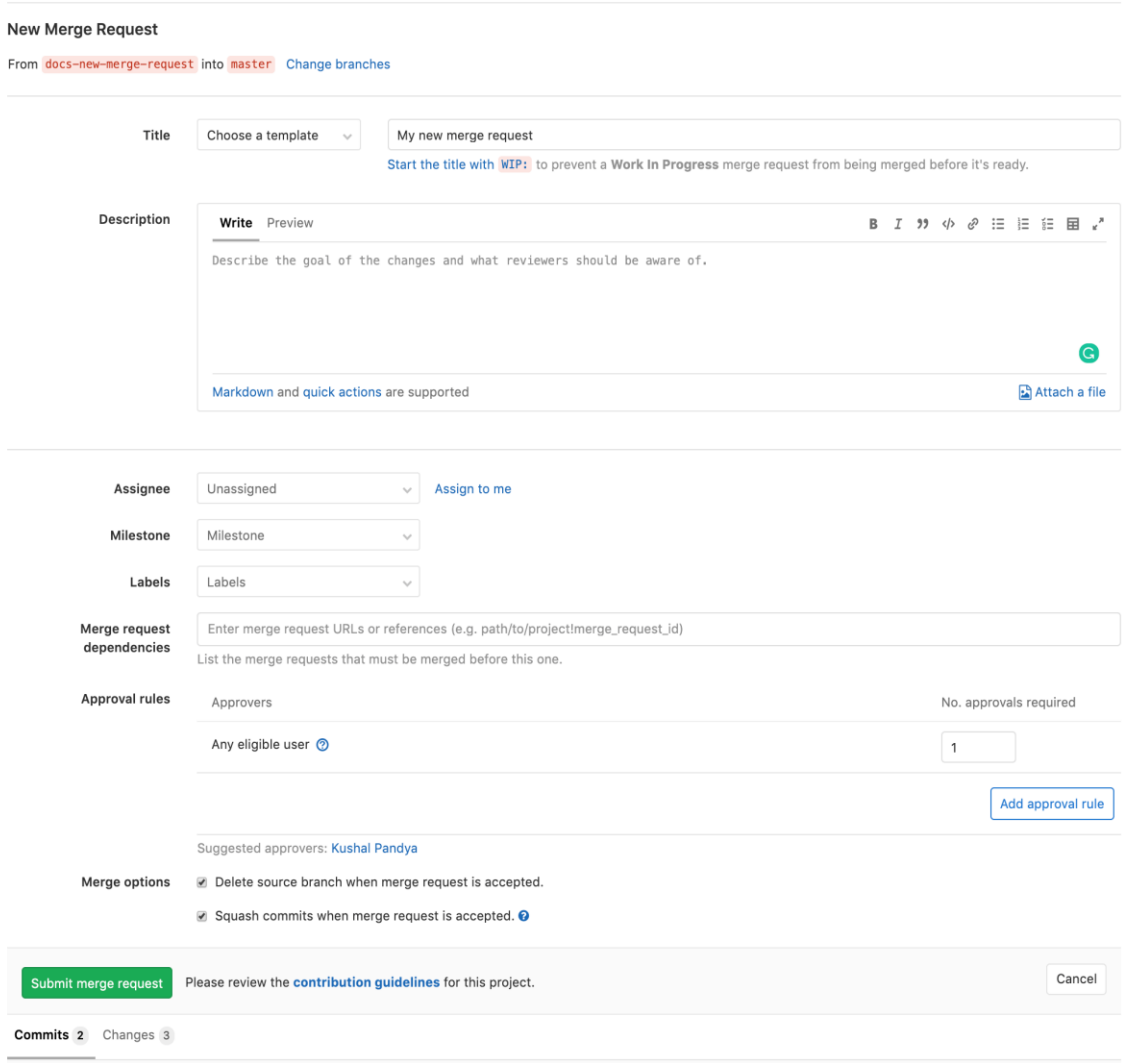
Alte concepte Git

- **Merge**
  - Integrează modificările dintr-un branch în altul, reunind contribuțiile dezvoltate separat
  - **git merge feature-login** (este folosită pentru a integra modificările din branch-ul feature-login în branch-ul pe care te afli în acel moment)

- **Pull Request / Merge Request**
  - Permite propunerea și revizuirea modificărilor înainte ca acestea să fie integrate într-un alt branch
  - GitHub: Pull Request (PR)
  - GitLab: Merge Request (MR)

- **Conflict**
  - Apare atunci când Git nu poate combina automat anumite modificări, de exemplu când două branch-uri modifică aceleași linii de cod
  - Dezvoltatorul trebuie să rezolve conflictul manual

- **.gitignore**
  - Fișier care specifică ce fișiere și directoare noi, neversionate, trebuie ignorate de Git, precum fișierele generate automat



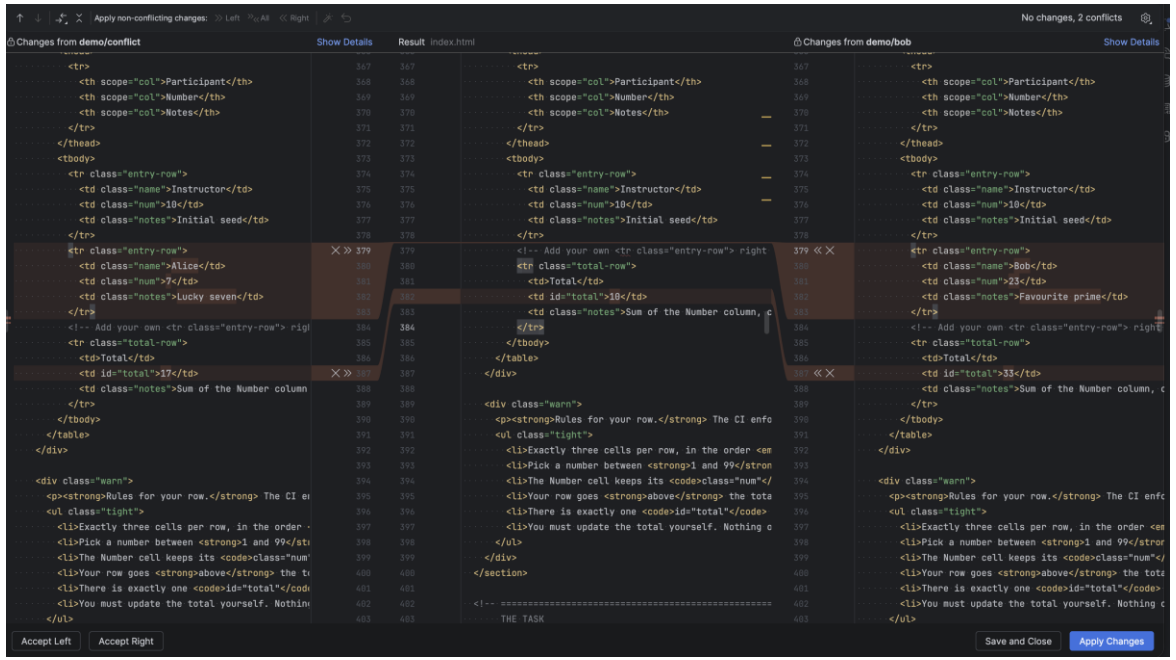
Conflicte in Git

➢ Conflicte în Git

Un conflict apare atunci când Git nu poate integra automat modificările din două ramuri ale istoricului, de exemplu când două branch-uri modifică aceeași parte a unui fișier.

➢ De ce apar?

- Doi dezvoltatori modifică aceleași linii de cod.
- Modificările sunt integrate prin merge sau pull, iar Git nu poate determina automat varianta corectă.



➢ Cum rezolvăm un conflict?

- Identificăm fișierele care conțin conflicte folosind git status.
- Deschidem fișierele și analizăm modificările.
- Alegem varianta corectă sau combinăm manual modificările.
- Eliminăm marcajele de conflict (<<<<<<<, =======, >>>>>>>).
- Adăugăm fișierele rezolvate în staging și finalizăm operația.

➢Rezolvarea conflictelor în IntelliJ IDEA

- Încercăm să integrăm modificările folosind Merge sau Pull.
- Deschidem instrumentul de rezolvare a conflictelor din IntelliJ IDEA (*Resolve Conflicts*).
- Comparăm cele două versiuni ale fișierului și alegem modificările pe care dorim să le păstrăm.
- Finalizăm rezolvarea și verificăm fișierul rezultat.
- Finalizăm operația Git prin commit, dacă este necesar.

© MHP Management- und IT-Beratung GmbH

Alte comenzi Git

- Cum actualizez repository-ul local fără a modifica fișierele?
  - git fetch
- Cum verific statusul branch-ului actual?
  - git status
- Cum adaug un fișier în zona de staging?
  - git add <file>
- Cum elimin un fișier din zona de staging?
  - git reset <file>
- Cum salvez schimbările ca un „checkpoint”?
  - git commit -m “<commit message>”
- Cum șterg un fișier și înregistrez ștergerea?
  - git rm <file>
- Cum vizualizez istoricul schimbărilor?
  - git log
- Cum văd istoricul unui anumit fișier?
  - git log --follow <file>
- Cum schimb branch-ul curent?
  - git checkout <branch> sau git switch <branch>
- Cum creez un branch?
  - git branch <nume branch>
- Cum creez și comut direct pe un branch nou?
  - git checkout -b <branch> sau git switch -c <branch>
- Cum aduc schimbările dintr-un alt branch în cel actual?
  - git merge <branch>
- Cum verific diferențele (schimbările) locale?
  - git diff
- Cum trimit schimbările pe branch-ul remote „origin”?
  - git push <origin> <branch>
- Cum aduc schimbările de pe branch-ul remote pe cel local?
  - git pull

© MHP Management- und IT-Beratung GmbH

Git Best practices

- **Commit-uri frecvente și logice**
  - Realizează commit-uri mici, care grupează modificări coerente și ușor de înțeles. Astfel, istoricul proiectului este mai clar, iar modificările nedorite pot fi anulate mai ușor
- **Mesaje de commit clare**
  - Folosește mesaje scurte și descriptive, care explică scopul modificărilor. Un istoric cu mesaje clare facilitează înțelegerea evoluției proiectului
- **Utilizarea branch-urilor**
  - Dezvoltă funcționalitățile noi și remedierile pe branch-uri separate, pentru a izola modificările de versiunea principală (main). Integrează-le după verificare și testare
- **Verificarea modificărilor înainte de commit**
  - Folosește zona de staging pentru a selecta modificările care vor fi incluse în commit. Verifică-le înainte de înregistrare pentru a evita includerea accidentală a unor fișiere sau modificări nedorite
- **Sincronizarea cu repository-ul remote**
  - Înainte de a integra modificări sau de a publica propriile commit-uri, verifică dacă branch-ul local este sincronizat cu cel remote
  - Integrează modificările colegilor pentru a reduce riscul conflictelor și al problemelor de sincronizare.
  - Comenzi: git fetch, git pull

Demo

DEMO

- GitHub - darius-f96/mhp-ubb-git-workshop · GitHub

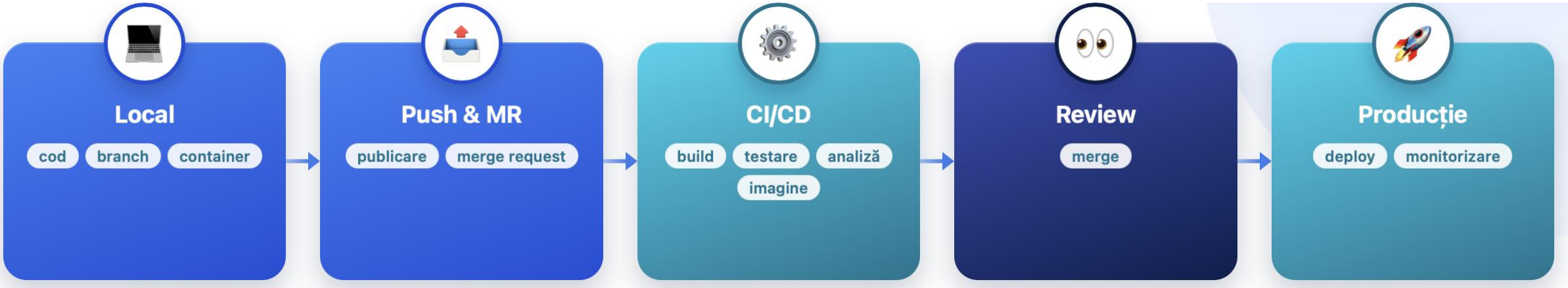
- Workshop Ledger — a Git exercise

Master the Stack



© MHP Management- und IT-Beratung GmbH

Master the Stack: Git & Docker & CI/CD: Modern Development Workflows



Docker

Aplicația împachetată, rulează oriunde

Ce veți învăța:

- Ce este un container și o imagine
- Cum scriem un Dockerfile
- Cum rulăm și publicăm imagini
- Cum ajunge imaginea de pe local în producție, trecând prin toți pașii intermediari de testare, analiză etc.

© MHP Management- und IT-Beratung GmbH


MISSION ACCOMPLISHED!
YOU ARE NOW GITHUB ARMY VETERANS!
THANK YOU FOR ENLISTING!
NOW GO FORTH, PUSH, AND COLLABORATE!